

mksglu/context-mode

Context Mode : 255 fichiers de tests vitest et une CI sur trois OS, mais des bugs ouverts de timeout et de budget et un seul mainteneur : une base testée, à qualifier zone par zone

Context window optimization for AI coding agents. Sandboxes tool output (98% reduction), persists session memory, and enforces routing across 17 platforms via MCP + hooks.

255

FICHIERS DE TEST
vitest

1

WORKFLOWS CI

4

BUGS ÉTIQUETÉS

119

ISSUES OUVERTES

☑ Que fait ce projet et quels sont ses parcours critiques ?

VERDICT

Quatre parcours à couvrir en priorité : interception puis indexation puis `ctx_search` , exécution en bac à sable dans 12 langages, compaction puis reprise, et `setup / doctor` sur 17 plateformes.

Context Mode est un serveur MCP plus des hooks qui interceptent les sorties d'outils des agents de code, les indexent dans SQLite FTS5, et n'exposent qu'un résumé recherchable ; il persiste aussi la session pour survivre à la compaction et impose un routage sur 17 plateformes.

Parcours critiques déduits du README et de CONTRIBUTING.md :

- ❑ Interception d'une sortie d'outil, indexation, recherche via `ctx_search` .
- ❑ Exécution de code en bac à sable dans 12 langages (`src/executor.ts`) et règles deny/allow (`src/security.ts`).
- ❑ Compaction puis reprise : hooks PreCompact, SessionStart, snapshot injecté.
- ❑ Installation et diagnostic par la CLI (`setup` , `doctor`) sur chaque plateforme.

Le repo compte 226 issues ouvertes toutes catégories.

☑ Comment est-il testé aujourd'hui ?

VERDICT

255 fichiers vitest lancés par `npm test` après un build automatique, mais aucune mesure de couverture dans les données relevées.

- ❑ `.agents/` config agents IA
- ❑ `.claude-plugin/` manifeste plugin Claude Code
- ❑ `.claude/` config locale Claude Code
- ❑ `.codex-plugin/` manifeste plugin Codex
- ❑ `.cursor-plugin/` manifeste plugin Cursor
- ❑ `.github/` 5 workflows GitHub Actions
- ❑ `.openclaw-plugin/` manifeste plugin OpenClaw
- ❑ `.pi/` extension pour Pi
- ❑ `bin/` script statusline.mjs
- ❑ `configs/` configs par plateforme
- ❑ `docs/` documentation
- ❑ `hooks/` hooks par plateforme, bundles session
- ❑ `scripts/` scripts build, install, version
- ❑ `skills/` skills pour agents
- ❑ `src/` TypeScript : serveur MCP, CLI, adaptateurs
- ❑ `tests/` tests vitest
- ❑ ... et 20 autres entrées à la racine du dépôt

Framework vitest, dossier `tests/`, 255 fichiers trouvés par recherche de code. `npm test` lance `vitest run`, précédé automatiquement d'un `npm run build`.

Les 20 premiers chemins montrent des sous-dossiers `tests/executor/`, `tests/core/` (deny-policy, routing), `tests/session/`, `tests/adapters/` (kiro, memory-conventions) et `tests/util/`, plus un `tests/mcp-integration.ts`. Un benchmark existe : `npx tsx tests/benchmark.ts`.

Données non relevées : couverture de code et liste complète des 255 fichiers ; à vérifier dans <https://github.com/mksglu/context-mode/tree/main/tests>.

☑ Que dit la CI et quand tourne-t-elle ?

VERDICT

Chaque push et pull request sur main et next passe typecheck, build, bundle et vitest sur ubuntu, macos et windows ; quatre workflows dont deux e2e restent non lus.

Un workflow lu, CI (`.github/workflows/ci.yml`), déclenché par workflow_dispatch, push et pull_request, sur les branches main et next.

Matrice ubuntu, macos et windows, fail-fast désactivé ; Node 22.5, Python 3.12, Go stable, Elixir 1.17 installés pour l'exécuteur polyglotte. Étapes : `npm install` , typecheck, build, bundle, assertions sur les bundles (garde-fou issue #511), puis vitest et enfin `npx tsx src/cli.ts doctor` en continue-on-error.

Quatre workflows non lus : bundle, openclaw-e2e, tier2-e2e-smoke, update-stats. Les deux e2e sont à examiner pour connaître la couverture bout en bout réelle.

☑ Où sont les bugs connus ?

VERDICT

4 issues étiquetées bug sur 119 ouvertes : le label ne suffit pas, partez des titres #947, #959, #950, #1022 et #1024.



119 issues ouvertes, 6 fermées sur 30 jours ; seules 11 sont étiquetées : 6 enhancement, 4 bug, 1 help wanted. Le tri par label est donc peu fiable, lire les titres.

Bugs les plus discutés :

- ❑ #950 : `ctx_stats` se contredit dans un même rendu et annonce 100 % d'économie sans aucun appel.
- ❑ #947 : le timeout de `ctx_batch_execute` ne borne pas l'indexation, un agent peut pendre des heures.
- ❑ #959 : sur l'adaptateur Pi, un `ctx_execute` bloqué ne peut pas être interrompu.
- ❑ #1024 : le nettoyage des bases périmées supprime des sessions vivantes mais inactives.
- ❑ #1022 : le snapshot de reprise ignore son budget d'octets, ~196 KB injectés au lieu de 2 KB.
- ❑ #911 et #946 : l'injection de prompt déclenche le classificateur de Claude Code.

☑ Comment reproduire et signaler un bug ici ?

VERDICT

Un rapport recevable = template d'issue, sortie de `bash scripts/ctx-debug.sh` , prompt exact et étapes de reproduction ; pas de SECURITY.md pour une faille, et 107 PR attendent déjà.

Le mainteneur demande trois choses : suivre les templates d'issue, lancer `bash scripts/ctx-debug.sh` et joindre sa sortie, écrire des tests pour tout correctif. Pour tester localement dans une vraie session : `npm run build` puis supprimer `server.bundle.mjs` , sinon vos changements ne sont jamais chargés.

Côté PR : 107 ouvertes, aucune fusionnée sur 30 jours dans les données relevées. Attendez-vous à une revue lente ; un rapport de bug précis avec sortie de debug a plus de valeur qu'une PR non testée.

☑ Quelles zones sont peu couvertes ?

VERDICT

Sans couverture mesurée, les trous visibles sont les timeouts, les budgets d'octets et les plateformes secondaires ; bus factor 1, donc vos tests sont la seule relecture croisée.

Donnée non relevée : mesure de couverture ; les zones ci-dessous sont déduites des issues et de la structure, à vérifier dans <https://github.com/mksglu/context-mode/tree/main/tests>.

- ☐ Timeouts et annulation (#947, #959) : les tests vus portent sur l'exécuteur et le bac à sable Windows, pas sur l'interruption.
- ☐ Budgets et statistiques (#950, #1022) : `tests/session/stats-output-format.test.ts` existe mais les bugs persistent.
- ☐ Plateformes secondaires : issue #45 cherche encore des testeurs sur 15 plateformes × 3 OS. Les adaptateurs kiro et memory-conventions ont des tests, les autres n'apparaissent pas dans les 20 chemins vus.
- ☐ Activité concentrée : 60 des commits sur 12 semaines viennent d'une seule personne, donc peu de relecture croisée.



Plan de test en 5 points

- 1 Environnement de qualification : `npm install` , `npm run build` , supprimer `server.bundle.mjs` , puis `npx tsx src/cli.ts doctor` ; à dérouler sur les trois OS de la matrice CI, car le doctor y tourne en continue-on-error et ne bloque rien.
- 2 Régression timeouts et annulation, risque haut : `ctx_batch_execute` doit s'arrêter au timeout même pendant l'indexation (#947) et un `ctx_execute` bloqué sur Pi doit répondre à Esc/Ctrl+C (#959).
- 3 Budgets et statistiques, risque haut : le snapshot de reprise doit rester sous son budget d'octets (#1022) et `ctx_stats` doit rester cohérent dans un même rendu, y compris sans appel `ctx_execute` (#950) ; point de départ `tests/session/stats-output-format.test.ts` .
- 4 Continuité de session, risque moyen : le nettoyage des bases périmées ne doit pas supprimer une session vivante mais inactive (#1024) ; vérifier la chaîne PostToolUse, PreCompact, SessionStart décrite dans CONTRIBUTING.md.
- 5 Plateformes, risque moyen : 17 plateformes annoncées, l'issue #45 cherche encore des testeurs sur 15 plateformes × 3 OS, et seuls les adaptateurs kiro et memory-conventions ont des tests dans les 20 chemins vus ; compléter par les workflows openclaw-e2e et tier2-e2e-smoke, et ajouter chaque test dans le fichier existant du domaine, jamais un nouveau.